Stage 2: Project Proposal

ALMANAC

A farm record book that knows what your crops are worth

Field	Value
Course	Database Management — 475/575
Team name	Lesotho
Captain	Dylan Montgomery — dcm103 — dcm103@uakron.edu
Members	Andrew Styles — as978@uakron.edu
Date	September 24, 2026

1. Project Title

Almanac: A Taxonomy-Linked Farm Record Book with Historical Market Pricing

2. Project Summary

Almanac is a web application in which a grower keeps a record book of what they planted, harvested, and sold, with every entry bound to a node in a 4,118-node biological taxonomy of plants.

Because entries are taxonomically identified, and because the same taxonomy indexes 47,516 real produce-auction observations collected from four independent Ohio market-report sources between 2019 and 2026, Almanac can answer questions an ordinary farm record book cannot: what a crop has historically fetched in a given week of the year, how reliably it has held that price, and — given a taxonomic family the grower is equipped to work — which crops in it have been the best bets for a particular planting window.

The application is built on a normalized relational database of twelve tables with fifteen foreign-key relationships, including two self-referential hierarchies. The reference data is already extracted, loaded, and integrity-checked.

3. Project Description

3.1 Application Domain

The domain is small-farm and market-garden record keeping, joined to agricultural commodity price history.

A grower — a market gardener, an orchardist, a CSA operator — needs to record events: a planting on a particular plot on a particular day, a harvest of a given quantity, a sale, a loss. Each of those events is about a specific crop. Separately, and today entirely disconnected from that record book, there exists a substantial body of public price data: produce auctions and USDA market-news services publish what each crop actually sold for, by the lot, day after day, season after season.

What we want to do is make those two bodies of information the same database, joined at the level of the organism. Every event a grower records names a taxon. Every price observation resolves, through a product lexicon, to a taxon. Once both sides share that key, the grower's own history and the region's price history become mutually queryable.

Concretely, Almanac will let a user:

- browse and search a deep plant taxonomy and a 217-entry product lexicon;
- maintain their own farm, plots, and crop catalog;
- create, edit, and delete event-log entries against a taxon and an optional plot;
- see the historical price curve for any crop they grow, by month, across seven seasons;
- roll prices up an arbitrary branch of the taxonomy, to any depth;
- rank candidate crops for a planting window by historical return and price stability.

3.2 Usefulness

The useful thing Almanac does is answer "what is this worth, and when?" inside the same tool where the grower already keeps their records — without asking them to look anything up, because the taxonomic identity they recorded is the join key.

The market for each half of this exists. The join does not.

Category	Examples	What they do — and do not do
Farm-management software	Tend, AgSquared, Farmbrite, Granular	Record plantings, harvests, tasks, and inventory. Strong record books. They carry no price history and no biological taxonomy — a crop is a free-text name the user typed.
Market price services	USDA AMS Market News, local produce-auction report pages and PDFs	Report what things sold for. Published as reports, not as a queryable corpus, and not connected to any individual operation's records.
Plant databases	USDA PLANTS, GBIF, Wikispecies	Authoritative taxonomy. No prices, no farm records, no notion of a user's own crop history.

How Almanac is different. It is the only one of the three that holds all three keys at once: a taxonomy deep enough to identify a crop precisely (twelve levels), a price corpus large enough to be statistically meaningful (47,516 observations over 560 trading days), and a user's own event log. Three questions follow from that and cannot be asked of any of the applications above:

- "What did this crop fetch in this week of the year, across seven seasons, and from which reporting source?" Requires the price corpus keyed to the same identity as the grower's record.

- “Price everything in this plant family.” Requires the taxonomy to be a real hierarchy that can be walked to arbitrary depth, not a flat list of crop names.
- “Of everything in this family I could plant in early May, what has held its price most reliably?” Requires all three together.

A grower deciding what to put in the ground in May currently answers that question from memory, from a neighbour, or from a price sheet they read and set aside. Almanac answers it from seven seasons of what the market actually paid, restricted to what they can actually grow.

3.3 Realness

Our data is real, it is already collected, and it is already loaded. It is not a dataset download.

The core of it is produce-auction market reports gathered and reconciled by one of our team members from four independent sources, plus a registry of Ohio farm operations. The extraction into a normalized relational form is complete: the tables below are populated and referential integrity has been verified, with zero orphaned foreign keys.

Table	Rows	What it is and where it came from
market_lot	47,516	Individual auction lots and daily product summaries: date, lot description, unit, quantity, and low / high / average price in cents. Four sources — County Line Produce Auction, Mt. Hope Auction, USDA AMS Mount Hope, and Geauga Growers Produce Auction — spanning 2019-05-03 to 2026-07-22 across 560 distinct trading days.
taxon	4,118	A plant-centered biological taxonomy, twelve levels deep, compiled from published taxonomic sources. Stored as a materialized path, so each node’s parent is derivable and the tree is walkable in SQL.
product	217	A lexicon collapsing thousands of inconsistent free-text auction descriptions onto canonical product slugs, each bound where possible to a taxon.
farm	238	Real Ohio farm operations, 131 of them with live registered web domains — e.g. trappfamilyfarm.com, bruntyfarms.com, cvcountryside.org.
region	2,818	Administrative geography, including all 88 Ohio counties as children of a single Ohio node, for county-level views.
observer	4	The provenance dimension: which source reported a price, its domain, and whether it reports per-lot or per-product-day. Every price in the system can be attributed.

We are being deliberate about the data’s imperfections rather than hiding them, because they are the substance of one of our advanced functions:

- 3,253 of the 47,516 lots carry a free-text description that no product slug yet claims — for example “10″ Hanging Basket” or “APPLES MCINTOSH YELLOW DELICIOUS.” Resolving them is Advanced Function 3.
- 53 rows were rejected outright by our CHECK constraints because their reported average price falls outside their own low–high range — genuine transcription errors in the upstream auction PDFs, such as “PUMPKINS Each, low $0.10, high $0.30, average $3.18.” We keep those rows in a quarantine table with the reason recorded rather than loosening a constraint that is correct.

The grower-authored half of the database — farms, plots, crop catalogs, and event-log entries — starts empty by nature. Per the project requirements we will have Almanac running and publicly reachable at least one week before the final presentation so that real users can enter real records, and we will notify the instructor when it is live.

3.4 Planned Functionality

3.4.1 Basic Functions

Insert, update, and delete.

Full create / edit / delete on everything a grower owns:

- event-log entries — a dated, quantified event naming a taxon, an event kind, an optional plot and an optional product. This is the primary write surface.
- plots — named ground belonging to a farm, with optional area.
- crop catalog entries — which taxa this farm grows.

Deletes are constrained rather than cascading blindly: deleting a plot leaves its events intact with a null plot reference, while a taxon that events depend on cannot be deleted at all. Those rules are enforced in the schema, not in the application.

Search and retrieval.

Several distinct queries, each answering a different question:

- Combined catalog search — one text box searching scientific names, common names, and product titles at once, ranked by how much price evidence each match has.
- Seasonal price curve — for one crop, the monthly floor, mean, and ceiling across all seven seasons. Drives a chart.
- Farm activity history — a farm’s events over a date range, filterable by event kind and crop.
- Source comparison — the same crop in the same period as reported by different market sources.

The required multi-table join. Our headline query joins six tables — the farm event log, the farm, the taxonomy, the event-kind vocabulary, the product lexicon, and the market-lot corpus — to answer: “for everything this farm has harvested,

what was that crop selling for at auction within thirty days of the harvest date?” It is written, executed, and returns correctly against the loaded data.

3.4.2 Advanced Functions

We propose three advanced functions. Two satisfy the 475 requirement; the third is included so the proposal also satisfies the 575 requirement of at least three, and because it is the function that most improves the data itself.

Advanced Function 1 — Recursive taxonomic price roll-up

The challenging problem. A user names one node of the taxonomy — a genus, a family, an order — and expects prices for everything beneath it. The taxonomy is a self-referential hierarchy twelve levels deep, and the number of levels below any given node is not known in advance. A conventional query with a fixed number of joins cannot express this; the obvious workaround is to make the application walk the tree with one query per level, which multiplies round trips and moves the work out of the database entirely.

Our solution. A recursive common table expression anchored on the named node, unioned with its children joined on the parent reference, descending until the subtree is exhausted — then left-joined through the product lexicon onto all 47,516 price observations and aggregated, in a single statement and a single round trip.

Why it is worth having. It turns the taxonomy from decoration into the query interface. Asked for the order Asparagales, the recursion walks 198 descendant nodes, finds the 6 that carry priced products, and returns 2,679 lots — onion alone being 1,382 lots at a $9.63 mean across seven seasons. No amount of flat crop-name matching produces that answer, because “onion,” “garlic” and “shallot” share no substring.

Advanced Function 2 — Crop decision engine

The challenging problem. “What should I plant in the first week of May?” The honest version of that question is hard: it needs a calendar window compared across multiple years rather than a single season, it has to weigh price against volatility (a crop with one spectacular week and five terrible ones is not a good bet), it must exclude anything with too little evidence to judge, and it has to be confined to crops the grower can actually work.

Our solution. A chain of common table expressions: a recursive subtree of the taxonomic branch the grower specifies, then all price observations falling inside the requested day-of-year window regardless of year, then per-product observation counts, distinct-year counts, means and spreads, then a stability score that discounts a crop’s mean price by its own volatility — ranked with a window function. Candidates must have at least twelve observations across at least two distinct years, so a single freak lot cannot top the ranking.

Why it is worth having. It converts seven seasons of raw price reports into an actionable, defensible ordering, restricted to what the grower can grow. Asked for late April rooted at flowering plants, it returns 25 ranked candidates led by strawberry, rose, and marigold — correct for an Ohio auction in that window, where nursery stock is what moves. Re-rooted at Asparagales, the same query returns two: garlic and asparagus. One parameter changes the question from “what should I plant” to “what should I plant in this family.”

One note on how we found the design. Our first version of this query ranked "Watering can" first, followed by metal art and planters — because these auctions sell merchandise alongside produce. The fix was not a list of things to exclude but a requirement that every candidate resolve to a taxon, which makes the filter and the question the same statement. We mention it because it is the clearest illustration of why the taxonomy is load-bearing rather than ornamental.

Advanced Function 3 — Taxon resolution ladder for free-text market labels

The challenging problem. 3,253 price observations carry a description written by whoever typed up that day's auction report — "APPLES MCINTOSH YELLOW DELICIOUS," "10″ Hanging Basket," "Beets Bunch Peck." Until each is bound to a taxon, its price is invisible to every query above. Naive substring matching is both wrong and dangerous here: "wild garlic" is not garlic, and a label matching two different taxa equally well must not be silently assigned to whichever the database happened to return first.

Our solution. A four-rung resolution ladder, tried in order of decreasing confidence: exact slug match on the normalized label; normalized-token match against product titles; token match against taxonomic common names; token match against scientific binomials. The rule that makes it correct is what happens at each rung — exactly one candidate binds it, zero candidates falls through to the next rung, and more than one candidate stops the ladder and reports the label as ambiguous, rather than falling through to a looser question that would produce a confident wrong answer. "Not found" and "ambiguous" are different outcomes with different remedies and are reported separately. Every proposed binding is confirmed by a human on a review screen before it is written.

Why it is worth having. It is the function that makes the rest of the corpus usable, and its effect is measurable: the share of the price corpus that is taxonomically resolved is a number that goes up. It is also generalizable in the way the assignment asks for — the ladder is driven by the contents of the taxonomy and lexicon tables, not by hard-coded rules about particular crops, so it keeps working as new sources and new labels arrive without the matching code being rewritten.

Anticipated platform. Python 3 with Flask and server-rendered templates, over a relational database accessed through parameterised SQL written by hand — no object-relational mapper, since the SQL is the substance of the work. The schema is currently implemented and verified on SQLite and is written to port to PostgreSQL or MySQL with only the auto-increment declaration and two date functions changing. We will confirm the required database platform with the instructor and finalise this in the Stage 3 development plan.

Current status at proposal time. The schema (12 tables, 15 foreign keys, 2 views), the reference-data loader, and all basic and advanced queries are written and have been executed against the loaded database; referential and structural integrity checks pass. The web application is the remaining work.